Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Алтайский государственный технический университет им. И.И. Ползунова»
Университетский технологический колледж
Кафедра Информационные системы в экономике

Отчет защищен с оценкой _____

Преподаватель С.В.Умбетов

«___» _____ 2026 г.

Отчет по лабораторной работе №
по дисциплине «»
«Устройство и функционирование интеллектуальных информационных систем»

ЛР 09.02.08.19.003

Студент группы 1ИИС-41 Н.Ю. Потапов
группа инициалы., фамилия
Преподаватель старший преподаватель, доцент С.В. Умбетов
должность, ученая степень инициалы., фамилия

Барнаул 2026

Цель работы: Изучить базовые этапы предобработки текста: токенизацию, очистку с помощью регулярных выражений, удаление стоп-слов, лемматизацию с помощью spaCy и NLTK, а также векторизацию текста методом TF-IDF.

Задачи к работе:

1. Установить и загрузить необходимые библиотеки (spaCy, русскоязычную модель ru_core_news_sm, NLTK со списком стоп-слов).
2. Сформировать исходный корпус текстов на русском языке.
3. Выполнить очистку текста от «шума» с помощью регулярных выражений (RegEx).
4. Провести токенизацию, удаление стоп-слов и лемматизацию с использованием spaCy.
5. Обернуть все этапы предобработки в единый пайплайн и обработать весь корпус.
6. Вручную рассчитать метрику TF-IDF для векторизации документов.
7. Выполнить самостоятельные задания по кастомизации стоп-слов, исследованию масштабируемости корпуса и модификации регулярных выражений.
8. Ответить на контрольные вопросы.

Задание принял:  Потапов Н.Ю.

Подпись ФИО

Ход работы:

Задание 1. Установка и загрузка библиотек

Ход выполнения: Для работы с русским языком были установлены библиотека spaCy и её официальная русскоязычная модель ru_core_news_sm, а также библиотека NLTK со списком стоп-слов русского языка.

```
!pip install spacy
!python -m spacy download ru_core_news_sm
```

Рисунок 1. Установка и загрузка необходимых библиотек

```
import re
import math
import spacy
import nltk
from nltk.corpus import stopwords
nltk.download('stopwords')
nlp = spacy.load("ru_core_news_sm")
print("\n\nComplete!")
```

```
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Package stopwords is already up-to-date!


Complete!
```

Рисунок 2. Загрузка списка стоп-слов NLTK

Задание 2. Формирование исходного корпуса текстов

Ход выполнения: Был определён список (корпус) из четырёх фраз на русском языке, объединённых общей тематикой (погода, прогулка, парк). Для демонстрации работы пайплайна был выбран второй документ в качестве тестового.

```
corpus = [
    "Сегодня была отличная погода, и мы пошли гулять в красивый парк!",
    "Ужасная погода... Весь день идёт сильный дождь, прогулка отменяется.",
    "В парке мы видели красивых птиц. Погода шепчет, солнце светит ярко.",
    "Дождь не прекращается! Сильный ветер сорвал все планы на вечер."
]

sample_text = corpus[1]
print(f"Исходный текст для анализов: {sample_text}")
```

Рисунок 3. Формирование исходного корпуса документов

Задание 3. Очистка текста с помощью регулярных выражений (RegEx)

Ход выполнения: Первый этап предобработки — удаление знаков препинания, цифр и приведение текста к нижнему регистру. Был применён шаблон r'[^а-яё\s]', который оставляет только русские буквы (включая «ё») и пробельные символы.

```
text_lowercase = sample_text.lower()
text_cleaned = re.sub(r'[^а-яё\s]', '', text_lowercase)
print(f"Текст после очистки RegEx: \n{text_cleaned}")
```

```
Текст после очистки RegEx:
ужасная погода весь день идёт сильный дождь прогулка отменяется
```

Рисунок 4. Очистка текста с помощью регулярных выражений

Задание 4. Токенизация, удаление стоп-слов и лемматизация (spaCy)

Ход выполнения: С помощью spaCy текст был разбит на токены, проведена лемматизация (приведение словоформ к начальной форме) и отфильтрованы стоп-слова из списка NLTK.

```
russian_stopwords = set(stopwords.words('russian'))
doc = nlp(text_cleaned)
filtered_tokens = []
for token in doc:
  lemma = token.lemma_
  if lemma not in russian_stopwords and lemma.strip():
    filtered_tokens.append(lemma)
print(f"Токены после лемматизации и удаления стоп-слов:\n{filtered_tokens}")
```

```
Токены после лемматизации и удаления стоп-слов:
['ужасный', 'погода', 'весь', 'день', 'идти', 'сильный', 'дождь', 'прогулка', 'отменяться']
```

Рисунок 5. Токенизация и лемматизация текста с помощью spaCy

Задание 5. Полная предобработка всего корпуса

Ход выполнения: Все предыдущие шаги были объединены в единую функцию pipeline_preprocess, после чего обработан весь корпус из четырёх документов.

```
def pipeline_preprocess(text):
  text = text.lower()
  text = re.sub(r'[^а-яё\s]', '', text)
  doc = nlp(text)
  return [lemma for token in doc if (lemma := token.lemma_.strip()) and lemma not in russian_stopwords]
processed_corpus = [pipeline_preprocess(text) for text in corpus]

for i, doc_tokens in enumerate(processed_corpus):
  print(f"Документ {i+1}: {doc_tokens}")
```

```
Документ 1: ['сегодня', 'отличный', 'погода', 'пойти', 'гулять', 'красивый', 'парк']
Документ 2: ['ужасный', 'погода', 'весь', 'день', 'идти', 'сильный', 'дождь', 'прогулка', 'отменяться']
Документ 3: ['парк', 'видеть', 'красивый', 'птица', 'погода', 'шепчет', 'солнце', 'светить', 'ярко']
Документ 4: ['дождь', 'прекращаться', 'сильный', 'ветер', 'сорвать', 'весь', 'план', 'вечер']
```

Рисунок 6. Полная предобработка всего корпуса текстов

Задание 6. Векторизация текста (расчёт TF-IDF вручную)

Ход выполнения: Для понимания математической сути TF-IDF метрика была рассчитана по базовым формулам без использования sklearn.

- TF (Term Frequency) = (количество вхождений слова в документе) / (всего слов в документе)
- IDF (Inverse Document Frequency) = ln(всего документов / документов со словом)
- TF-IDF = TF × IDF

```
def vocab():
  vocabulary = sorted(list(set([word for doc in processed_corpus for word in doc])))
  print(f"Уникальный словарь корпуса (всего слов: {len(vocabulary)}): {vocabulary}")
vocab()
```

Рисунок 7. Формирование уникального словаря корпуса

```
def calculate_tfidf(word, current_doc, all_docs):
  tf = current_doc.count(word) / len(current_doc) if len(current_doc) > 0 else 0

  docs_with_word = sum(1 for d in all_docs if word in d)
  idf = math.log(len(all_docs) / docs_with_word)
  return round(tf * idf, 4)

print("\nВекторы документов (TF-IDF числовые представления):")
def calc_vector():
  for i, doc in enumerate(processed_corpus):
    vector = []
    for word in vocabulary:
      tfidf_value = calculate_tfidf(word, doc, processed_corpus)
      vector.append(tfidf_value)
    print(f"Вектор Документа {i+1}: {vector}")
calc_vector()
```

```
Уникальный словарь корпуса (всего слов: 26): ['весь', 'ветер', 'вечер', 'видеть', 'гулять', 'день', 'дождь', 'идти', 'красивы

Векторы документов (TF-IDF числовые представления):
Вектор Документа 1: [0.0, 0.0, 0.0, 0.0, 0.198, 0.0, 0.0, 0.0, 0.099, 0.198, 0.0, 0.099, 0.0, 0.0411, 0.198, 0.0, 0.0, 0.0, 0
Вектор Документа 2: [0.077, 0.0, 0.0, 0.0, 0.0, 0.154, 0.077, 0.154, 0.0, 0.0, 0.154, 0.0, 0.0, 0.032, 0.0, 0.0, 0.154, 0.0,
Вектор Документа 3: [0.0, 0.0, 0.0, 0.154, 0.0, 0.0, 0.0, 0.0, 0.077, 0.0, 0.0, 0.077, 0.0, 0.032, 0.0, 0.0, 0.0, 0.154, 0.15
Вектор Документа 4: [0.0866, 0.1733, 0.1733, 0.0, 0.0, 0.0, 0.0866, 0.0, 0.0, 0.0, 0.0, 0.0, 0.1733, 0.0, 0.0, 0.1733, 0.0, 0
```

Рисунок 8. Расчёт векторов TF-IDF для документов

Задание 7. Глубокий анализ и кастомизация стоп-слов

7.1. Вывод количества и списка стоп-слов

Ход выполнения: Написан код, выводящий общее количество стоп-слов и отсортированный по алфавиту список.

```
def stopwords_count(current_words):
  count_stopwords = len(current_words)
  print(f"Кол-во русских стоп-слов: {count_stopwords}")
  sort_stopwords = sorted(current_words)
  print(f"Стоп-слова:\n{sort_stopwords}\n")

stopwords_count(russian_stopwords)
```

Рисунок 9. Код, выводящий общее кол-во стоп-слов

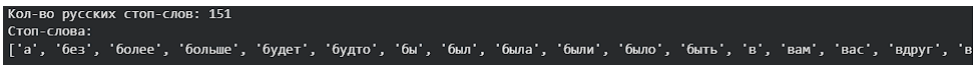


Рисунок 10. Вывод списка русских стоп-слов

7.2. Классификация 5 стоп-слов

№	Слово	Часть речи	Лингвистическая роль
1	он	Личное местоимение	Указывает на субъект действия, не несёт самостоятельной семантики
2	в	Предлог	Выражает пространственные/временные отношения
3	и	Союз (сочинительный)	Связывает однородные члены предложения
4	не	Частица (отрицательная)	Выражает отрицание
5	этот	Указательное местоимение	Указывает на близкий предмет

Все эти слова выполняют преимущественно синтаксическую функцию и обладают низкой информационной ценностью для задач тематического моделирования, поэтому их удаление оправдано.

7.3. Добавление кастомных стоп-слов и повторная обработка корпуса

Ход выполнения: В множество стоп-слов добавлены слова «сегодня» и «весь», которые часто встречаются в нашем корпусе. Корпус обработан заново с использованием обновлённого списка.

```
new_stopwords = russian_stopwords | {'сегодня', 'весь'}
print("Добавлены слова")
stopwords_count(new_stopwords)

def pipeline_preprocess_temp(text, stopword):
  text = text.lower()
  text = re.sub(r'[^а-яё\s]', '', text)
  doc = nlp(text)
  return [lemma for token in doc if (lemma := token.lemma_.strip()) and lemma not in stopword]
processed_corpus_temp = [pipeline_preprocess_temp(text, new_stopwords) for text in corpus]

for i, doc_tokens in enumerate(processed_corpus_temp):
  print(f"Документ {i+1}: {doc_tokens}")
```

Рисунок 11. Добавление кастомных стоп-слов "сегодня" и "весь"

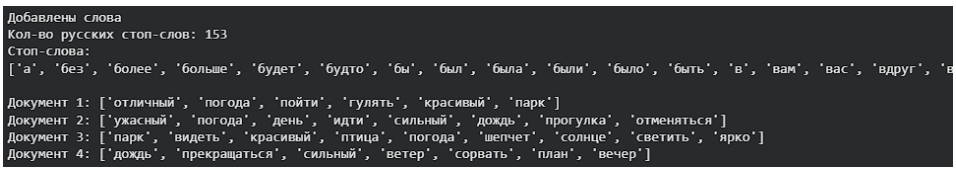


Рисунок 12. Обработка корпуса с обновлённым списком стоп-слов

Сравнение с исходным результатом:

- В Документе 1 исчезло слово «сегодня» (было 7 токенов, стало 6).
- В Документе 2 исчезло слово «весь» (было 9 токенов, стало 8).
- В Документе 4 также исчезло слово «весь» (было 8 токенов, стало 7).

Кастомные стоп-слова успешно удалены из корпуса. Общее количество стоп-слов увеличилось с 151 до 153.

Задание 8. Исследование масштабируемости корпуса и динамики векторов

8.1. Добавление пятого документа в корпус

Ход выполнения: В корпус добавлен пятый документ, содержащий как новые слова («улица», «поют», «прекрасная», «поэтому»), так и уже встречавшиеся («птицы», «погода», «сегодня», «гуляем»).

```
vocab()
for i, doc_tokens in enumerate(processed_corpus):
  print(f"Документ {i+1}: {doc_tokens}")
calc_vector()
corpus.append("На улице поют птицы, погода прекрасная! Поэтому сегодня мы гуляем")
processed_corpus = [pipeline_preprocess(text) for text in corpus]
print("Добавлен новый документ\n")
vocab()
for i, doc_tokens in enumerate(processed_corpus):
  print(f"Документ {i+1}: {doc_tokens}")
calc_vector()
```

Рисунок 13. Добавление пятого документа в корпус

8.2. Сравнение размера словаря и длины векторов

До добавления документа:

- Уникальный словарь: 26 слов
- Длина вектора каждого документа: 26 элементов

После добавления документа:

- Уникальный словарь: 30 слов (добавились: «петь», «поэтому», «прекрасный», «улица»)
- Длина вектора каждого документа: 30 элементов

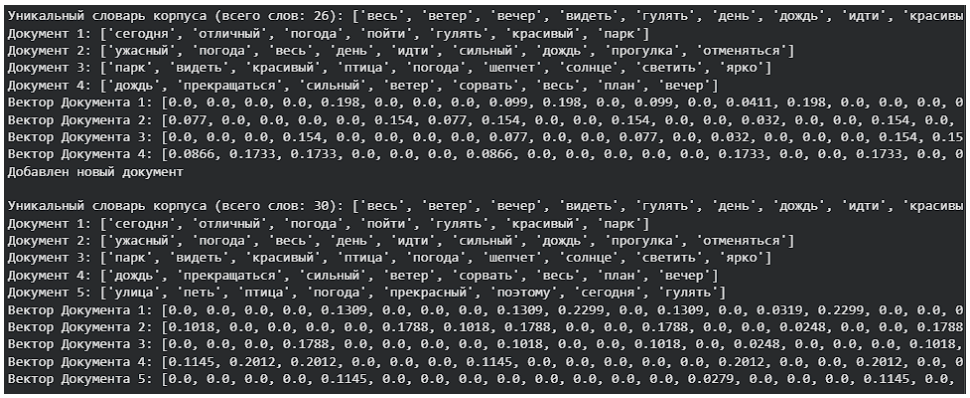


Рисунок 14. Сравнение размера словаря до и после добавления документа

Размер словаря увеличился на 4 слова (с 26 до 30), что привело к увеличению длины каждого вектора на 4 позиции. Все существующие векторы были пересчитаны, так как изменился IDF для слов, присутствовавших в новом документе.

Проанализированы значения TF-IDF для новых слов в пятом документе и сравнены с весами слов, дублировавшихся из прошлых текстов.

Расчёт для слова «улица» (новое, встречается только в документе 5):

- TF = 1/8 = 0.125 (1 вхождение в документе из 8 токенов)
- IDF = ln(5/1) = ln(5) ≈ 1.609 (встречается в 1 из 5 документов)
- TF-IDF = 0.125 × 1.609 ≈ 0.201

Расчёт для слова «погода» (встречается во всех 5 документах):

- TF = 1/8 = 0.125
- IDF = ln(5/5) = ln(1) = 0
- TF-IDF = 0.125 × 0 = 0

Расчёт для слова «сегодня» (встречается в 2 документах):

- TF = 1/8 = 0.125
- IDF = ln(5/2) = ln(2.5) ≈ 0.916
- TF-IDF = 0.125 × 0.916 ≈ 0.115

Вес новых уникальных слов («улица», «петь», «прекрасный») оказался выше, чем у слов, дублировавшихся из прошлых текстов, потому что компонент IDF обратно пропорционален частоте встречаемости слова в корпусе. Чем реже слово встречается в документах, тем выше его IDF и, соответственно, TF-IDF. Слово «погода», присутствующее во всех документах, получило IDF = 0 и полностью обесценилось — оно не несёт дифференцирующей информации. Это и есть смысл метрики TF-IDF: она повышает вес редких, специфичных терминов и понижает вес общеупотребительных.

Задание 9. Модификация регулярных выражений под сохранение числовых данных

9.1. Добавление документа с числовыми метриками и изменение RegEx

Ход выполнения: В корпус добавлен документ «В 2026 году мы выполнили 3 проекта и обработали 15 документов!». Регулярное выражение изменено с r'[^a-яё\s]' на r'[^a-яё0-9\s]' — добавлен диапазон цифр 0-9.

```
corpus_with_numbers = corpus.copy()
corpus_with_numbers.append("В 2026 году мы выполнили 3 проекта и обработали 15 документов!")

def pipeline_preprocess_with_numbers(text):
    text = text.lower()
    text = re.sub(r'[^a-яё0-9\s]', '', text)
    doc = nlp(text)
    return [lemma for token in doc if (lemma := token.lemma_.strip()) and lemma not in russian_stopwords]

processed_corpus_numbers = [pipeline_preprocess_with_numbers(text) for text in corpus_with_numbers]
print(f"\nИсходный текст: {corpus_with_numbers}")
for i, doc_tokens in enumerate(processed_corpus_numbers):
  print(f"Документ {i+1}: {doc_tokens}")
```

Рисунок 15. Модификация регулярного выражения для сохранения числовых данных и обработка текса с их сохранением

```
Исходный текст: ['Сегодня была отличная погода, и мы пошли гулять в красивый парк!', 'Ужасная погода... Весь день идёт сильны
Документ 1: ['сегодня', 'отличный', 'погода', 'пойти', 'гулять', 'красивый', 'парк']
Документ 2: ['ужасный', 'погода', 'весь', 'день', 'идти', 'сильный', 'дождь', 'прогулка', 'отменяться']
Документ 3: ['парк', 'видеть', 'красивый', 'птица', 'погода', 'шепчет', 'солнце', 'светить', 'ярко']
Документ 4: ['дождь', 'прекращаться', 'сильный', 'ветер', 'сорвать', 'весь', 'план', 'вечер']
Документ 5: ['улица', 'петь', 'птица', 'погода', 'прекрасный', 'поэтому', 'сегодня', 'гулять']
Документ 6: ['2026', 'год', 'выполнить', '3', 'проект', 'обработать', '15', 'документ']
```

Рисунок 16. Результат обработки

9.2. Анализ токенизации чисел моделью spaCy

Обновлённый текст прогнан через модель spaCy для анализа того, как токенизатор обработал комбинацию цифр и букв.

Результаты анализа:

- Строка «2026» стала отдельным токеном — токенизатор spaCy корректно выделяет последовательности цифр как самостоятельные единицы.
- Строки «3» и «15» также стали отдельными токенами.
- Лемма, присвоенная числительным: «2026» → «2026», «3» → «3», «15» → «15». Для числительных лемма совпадает с исходной формой, так как у чисел нет флективных изменений в том же смысле, что у существительных или глаголов.

Вывод о практической применимости сохранения цифр

В задачах, где сохранение цифр критически важно:

- Анализ финансовых документов и чеков — суммы, номера счетов, даты платежей несут ключевую смысловую нагрузку.
- Парсинг адресов и геоданных — номера домов, почтовые индексы, координаты.
- Медицинские записи — дозировки, даты приёмов, показатели анализов.
- Техническая документация — версии ПО, артикулы, серийные номера.
- Анализ новостей и отчётов — годы, статистические показатели, проценты.

В задачах, где цифры — избыточный «шум»:

- Тематическое моделирование художественной литературы — числа редко несут тематическую нагрузку.
- Анализ тональности отзывов — конкретные цифры (например, «купил 2 штуки») обычно не влияют на эмоциональную окраску.

- Кластеризация новостных статей по темам — годы и числа могут создавать ложные кластеры.
- Поиск семантической близости документов — числа увеличивают размерность пространства признаков без добавления смысла.

Вывод:

В ходе лабораторной работы я изучил и практически освоил полный пайплайн предобработки текстовых данных на русском языке. Научился очищать текст от шума с помощью регулярных выражений, проводить токенизацию и лемматизацию с использованием библиотеки spaCy, удалять стоп-слова из списка NLTK и кастомизировать этот список под конкретную задачу. Вручную рассчитал метрику TF-IDF, что позволило глубоко понять математическую природу векторизации: редкие слова получают высокий вес за счёт компонента IDF, а общеупотребительные — обесцениваются. Исследовал масштабируемость корпуса: добавление новых документов увеличивает словарь и длину векторов, пересчитывая IDF для всех терминов. Модифицировал регулярные выражения для сохранения числовых данных и проанализировал, как spaCy токенизирует и лемматизирует числительные. Приобретённые навыки являются фундаментом для решения реальных задач NLP: тематического моделирования, классификации текстов, анализа тональности и информационного поиска.

Ссылка на мой репозиторий с работами

Ссылка на блокнот данной лабораторной

Ответы на контрольные вопросы:

1. Какова роль регулярных выражений (RegEx) при предварительной очистке текстового шума?

RegEx позволяют гибко и эффективно фильтровать неинформативные символы: знаки препинания, веб-теги, системные символы, цифры (если они не нужны). Они нормализуют регистр, удаляют избыточные пробелы и подготавливают текст к токенизации, работая на основе теории конечных автоматов.

2. К каким ошибкам в анализе может привести избыточное удаление спецсимволов и дефисов?

Удаление дефисов разрушает составные слова: «e-mail» превратится в «email» или «e mail», «он-лайн» — в «онлайн» или два отдельных токена. Это искажает семантику, создаёт ложные токены и снижает качество лемматизации. В некоторых случаях (например, «из-за») удаление дефиса меняет смысл.

3. Почему приведение текста к нижнему регистру (lowercase) является обязательным шагом перед токенизацией?

Без lowercase слова «Парк», «парк» и «ПАРК» будут восприняты как три разных токена, что искусственно раздувает словарь, занижает TF и искажает IDF. Приведение к нижнему регистру обеспечивает единообразие и корректный подсчёт частот.

4. Как настроить регулярное выражение, чтобы оно сохраняло в тексте и буквы, и важные цифровые данные?

Нужно добавить диапазон цифр в исключающий класс символов: r'[^а-яё0-9\s]'. Этот шаблон оставит русские буквы (включая «ё»), арабские цифры и пробельные символы, удалив всё остальное. Для сохранения дат в формате «01.01.2026» можно расширить шаблон до r'[^а-яё0-9.\s]'.

5. В каких задачах обработки текста нельзя удалять знаки препинания и смайлики (emoji)?

Анализ тональности (sentiment analysis) — смайлики и восклицательные знаки несут эмоциональную нагрузку.

Определение границ предложений — точки и вопросительные знаки нужны для сегментации.

Анализ диалогов и чатов — эмодзи часто заменяют слова.

Стилометрия и авторство текста — пунктуационные паттерны уникальны для авторов.

6. В чём заключается главное различие между процессами лемматизации и стемминга?

Лемматизация приводит слово к его словарной форме (лемме) с учётом контекста и части речи — результат всегда является реальным словом языка. Стемминг отсекает окончания по эвристическим правилам, получая «псевдооснову» (стем), которая может не быть существующим словом (например, «бежа» от «бежать»).

7. Почему для флективных языков (таких как русский) лемматизация эффективнее стемминга?

Русский язык имеет богатую флективную систему: одно слово может иметь десятки форм (падежи, числа, рода, времена). Стемминг не учитывает контекст и часть речи, поэтому часто даёт некорректные основы. Лемматизация, опираясь на морфологический анализ и POS-теги, точно определяет начальную форму, что критически важно для корректного подсчёта частот и векторизации.

8. Как алгоритм токенизации определяет границы слов при наличии сокращений (например, «т.д.», «и т.п.»)?

Современные токенизаторы (включая spaCy) используют комбинацию правил и статистических моделей. Они распознают распространённые сокращения как единые токены, не разрывая их по точкам. Для неизвестных сокращений применяются эвристики: точка между заглавными буквами или в конце короткой последовательности часто не считается границей предложения.

9. По какому принципу слова формируют список стоп-слов в библиотеках NLTK или spaCy?

Список стоп-слов формируется на основе частотного анализа больших корпусов текстов. В него включаются наиболее употребительные слова, выполняющие преимущественно синтаксическую функцию (предлоги, союзы, частицы, местоимения) и не несущие самостоятельной семантической нагрузки. Список может дополняться вручную под конкретную предметную область.

10. Что произойдёт с качеством анализа тональности отзывов, если удалить частицу «не» как стоп-слово?

Качество резко ухудшится. Частица «не» меняет полярность высказывания: «хороший» → «не хороший». При её удалении оба отзыва будут классифицированы как положительные, что приведёт к систематической ошибке. В задачах анализа тональности «не» обычно сохраняют или используют биграммы («не хороший» как единый токен).

11. Какой физический и математический смысл имеет метрика TF (Term Frequency)?

Физический смысл: TF показывает, насколько часто слово встречается в конкретном документе относительно его длины — это мера «локальной важности» термина. Математически: TF = (число вхождений слова в документе) / (общее число слов в документе). Нормировка на длину документа предотвращает завышение веса слов в длинных текстах.

12. Зачем при расчёте метрики IDF (Inverse Document Frequency) используется операция логарифмирования?

Логарифм сглаживает резкие перепады значений. Без логарифма отношение N/df может принимать очень большие значения для редких слов, что приводит к числовой нестабильности и доминированию единичных терминов. Логарифм обеспечивает плавное нарастание веса: слово, встречающееся в 1 из 1000 документов, не будет в 1000 раз «важнее» слова из 1 из 10 документов.

13. Чему будет равен коэффициент TF-IDF для слова, которое встречается абсолютно во всех документах корпуса?

IDF = ln(N/N) = ln(1) = 0. Следовательно, TF-IDF = TF × 0 = 0. Такие слова полностью обесцениваются, так как не позволяют различать документы — они присутствуют везде и не несут дифференцирующей информации.

14. Какая часть метрики TF-IDF отвечает за локальную важность слова, а какая — за его глобальную уникальность?

TF отвечает за локальную важность — насколько слово значимо внутри конкретного документа.

IDF отвечает за глобальную уникальность — насколько слово редко встречается во всём корпусе и поэтому информативно.

15. Как длина текста влияет на показатель TF и почему длинные документы могут искусственно занижать вес слов?

TF нормируется на общее число слов в документе. В длинном документе одно и то же слово, встречаясь, скажем, 5 раз, будет иметь меньший TF, чем в коротком документе, где оно встречается те же 5 раз. Это корректное поведение: в длинном тексте 5 вхождений менее значимы, чем в коротком. Однако если слово действительно ключевое для длинного документа, его TF может оказаться заниженным — для борьбы с этим используют логарифмическую нормировку TF: TF = 1 + ln(count).

16. Что происходит с размером итоговой матрицы TF-IDF при добавлении в корпус новых уникальных слов?

Матрица TF-IDF имеет размерность (число документов × размер словаря). При добавлении новых уникальных слов размер словаря увеличивается, следовательно, увеличивается количество столбцов матрицы. Все существующие векторы документов расширяются нулевыми значениями для новых слов, а IDF пересчитывается для всех терминов.

17. Что такое «разреженность» матрицы (Sparsity) и почему в итоговой таблице TF-IDF преобладают нули?

Разреженность — это доля нулевых элементов в матрице. В TF-IDF матрице преобладают нули, потому что каждое слово встречается лишь в небольшой части документов корпуса. При большом словаре (тысячи терминов) и относительно малом числе документов большинство ячеек будут нулевыми. Высокая разреженность — это норма для текстовых данных и учитывается при выборе алгоритмов (например, используют разреженные матрицы scipy.sparse для экономии памяти).

18. Как ошибка определения части речи в библиотеке spaCy может повлиять на итоговый результат лемматизации?

Лемматизация в spaCy опирается на POS-тег: одно и то же слово может иметь разные леммы в зависимости от части речи (например, «стекло» как существительное → «стекло», как глагол → «стечь»). Если POS-тег определён неверно, лемматизатор выберет неправильную парадигму склонения/спряжения, и слово будет приведено к неверной начальной форме. Это исказит частотный анализ и векторизацию.

19. В чём плюсы и минусы перехода от анализа одиночных слов к анализу словосочетаний (биграмм и триграмм)?

Плюсы:

Сохраняется контекст и устойчивые выражения («не хороший», «Нью-Йорк», «машинное обучение»).

Снижается полисемия: «горячий чай» и «горячий спор» становятся разными токенами.

Повышается качество тематического моделирования и классификации.

Минусы:

Резко растёт размер словаря (комбинаторный взрыв).

Увеличивается разреженность матрицы.

Требует больше вычислительных ресурсов и данных для обучения.

Редкие n-граммы могут не встретиться в обучающей выборке.

20. Что произойдёт, если в векторизатор TF-IDF передать новое слово, которого вообще не было в исходном обучающем словаре?

В классическом TF-IDF (например, sklearn TfidfVectorizer) новое слово будет проигнорировано — оно не имеет столбца в матрице и не получит векторного представления. Это проблема «out-of-vocabulary» (OOV). Для решения используют: (1) обновление словаря и пересчёт IDF, (2) хеширование признаков (feature hashing), (3) переход к эмбеддингам (Word2Vec, BERT), которые могут работать с неизвестными словами через подсловные токены.